

various converters for TensorFlow, Keras, CoreML, Scikit Learn, XGBoost, LibSVM and NCNN(Tencent). The execution runtimes supported by ONNX are NVIDIA, Qualcomm, Bitmain, Tencent, Vespa, Windows, Synopsys, Ceva, Mace, Habana and, recently, Onnx Runtime. There are also compilers for ONNX models from Intel AI, Skymizer and TVM. NETRON and VisualDL help ONNX to visualise and manipulate graphs.

With the wide range of frameworks, converters, runtimes, compilers and visualisers, ONNX unlocks the dependencies and allows developers to make their own choices regarding frameworks and tools. ONNX has two variants. The base variant is for machine learning based on neural networks. Another variant, ONNX-ML, adds additional operators and data types for classical machine learning algorithms.

More details about ONNX Intermediate Representation (IR), operator information and graph utilities can be found at <https://github.com/onnx/onnx>.

## Setting it up

ONNX is released as a Python package and can be installed and verified. ONNX graph representation is based on protobuf; hence, protobuf is a dependency for ONNX. On Ubuntu Linux, the following command can install protobuf dependencies:

```
test@test-pc:~$ sudo apt-get install protobuf-compiler
libprotoc-dev
```

The ONNX Python package can be installed and verified from the Python package manager as shown below:

```
test@test-pc:~$ pip3 install onnx
test@test-pc:~$ python3 -c "import onnx"
```

ONNX is just a graphical representation and when it comes to executing an ONNX model, we still need a back-end. In this example, we use the TensorFlow back-end to run the ONNX model and hence install the package as shown below:

```
test@test-pc:~$ pip3 install onnx_tf
test@test-pc:~$ python3 -c "from onnx_tf.backend import
prepare"
```

Now we have the setup ready to build a model in ONNX and execute it in the TensorFlow back-end.

Alternatively, ONNX is also published as Docker images for CPU and GPU environments. Those who are familiar with Docker can use the following command to set up the ONNX Docker environment:

```
docker run -it --rm onnx/onnx-docker:cpu /bin/bash
```



Figure 1: The ONNX ecosystem

Use the following command for environments with GPU support. This option requires NVIDIA-Docker as a prerequisite.

```
nvidia-docker run -it --rm onnx/onnx-docker:gpu /bin/bash
```

## An example

This simple example demonstrates how to build a simple graph and execute it on the TensorFlow back-end. Copy the following snippet of code to add two tensors as *test.py*:

```
# Imports from ONNX
from onnx import helper, TensorProto
from onnx_tf.backend import prepare
import numpy as np

# Input shape and dtype
in_shape = (2, 3)
dtype = "float32"
out_shape = in_shape

# Make use of helper to construct a node to produce "out" by
adding "in1" and "in2".
z = helper.make_node("Add", ['in1', 'in2'], ['out'])

# Make a graph that produce z from inputs named "in1" and
"in2"
graph = helper.make_graph([z],
                           '_test',
                           inputs = [helper.make_tensor_value_info("in1",
                               TensorProto.FLOAT, list(in_shape)),
                               helper.make_tensor_value_info("in2",
                               TensorProto.FLOAT, list(in_shape))],
                           outputs = [helper.make_tensor_value_info("out",
                               TensorProto.FLOAT, list(out_shape))])

# Now make an ONNX model from graph.
model = helper.make_model(graph, producer_name='_test')
```